

MILESTONE 3 · MODEL ARCHITECTURE DESIGN

AI-Powered Driver Wellness & Safety Monitoring System

Five deep-learning modules — architectures selected, justified and specified end-to-end, converging on a single Risk Fusion Engine that scores driver wellness in real time.



Kushagra · Shiwani · Shubham · Sohini · Ravina

Why Driver Monitoring, Why Now

BACKGROUND & MOTIVATION

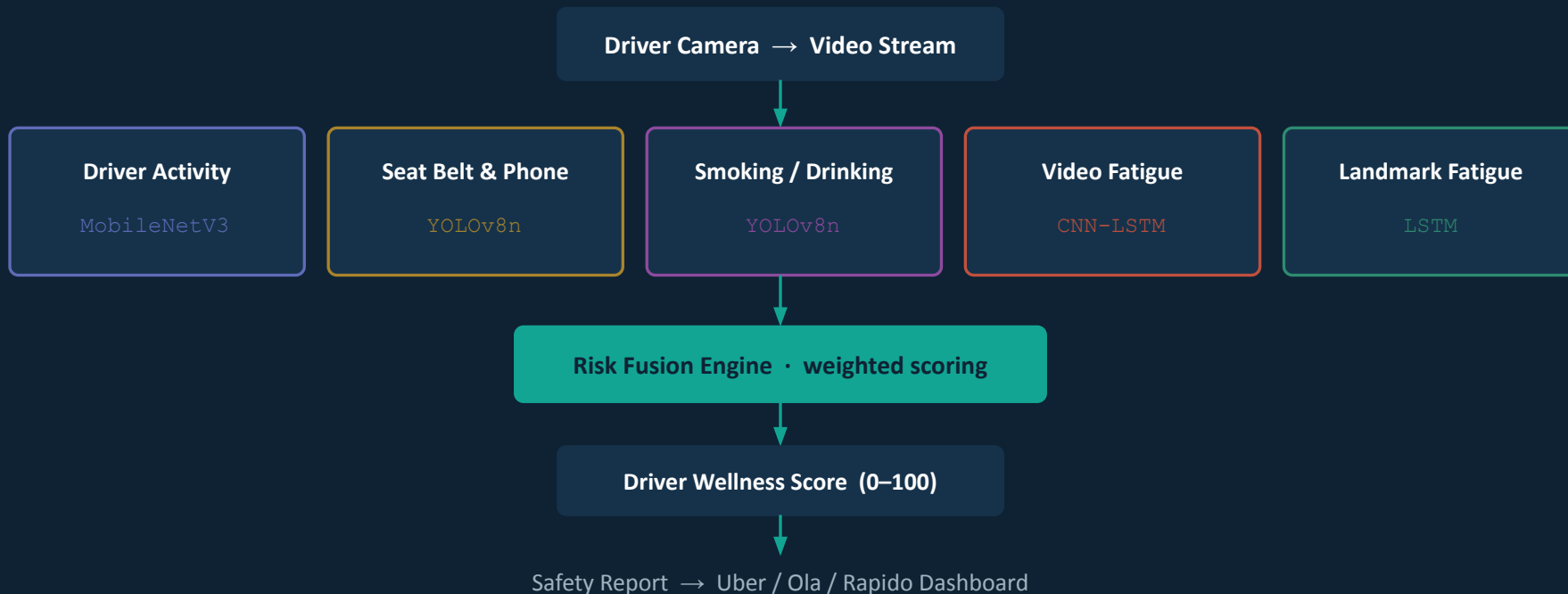
- Fatigue and distraction remain leading causes of commercial-fleet road accidents
- Fleet platforms (Uber, Ola, Rapido) have no continuous, objective view of driver state
- A single cabin camera can observe all of it — if the models are cheap enough to run on the edge

One camera. Five signals. One score.

MILESTONE 3 OBJECTIVES

- Select and justify an architecture per module
- Fix input / output specifications
- Choose loss functions and evaluation metrics
- Plan hyperparameters and estimate compute
- Design the end-to-end inference pipeline
- Make the system implementation-ready for M4

Five Models, One Decision



Five Modules, One System

MEMBER	MODULE	DATASET	SELECTED MODEL	INPUT
 Kushagra	Video-Based Fatigue Detection	UTA-RLDD	CNN-LSTM	(16, 224, 224, 3)
 Shiwani	Landmark Temporal Fatigue	YawDD	LSTM	(30, 5)
 Shubham	Driver Activity Classification	AUC Distracted Driver	MobileNetV3	(224, 224, 3)
 Sohini	Seat Belt & Phone Detection	DMS Dataset	YOLOv8n	640 × 640
 Ravina	Smoking & Drinking Detection	Merged YOLO set	YOLOv8n	640 × 640

How Every Architecture Was Chosen

01

Candidate set

2–4 architectures
shortlisted per task
from current literature

02

Constraint filter

Real-time latency,
edge memory, and
dataset size applied as
hard filters

03

Evidence

Baseline runs or
published behaviour
compared on the
module's own data

04

Full spec

I/O shape, loss,
metrics,
hyperparameters and
compute recorded

Shared across all five modules(Except Fatigue Detection): PyTorch / Ultralytics · Adam or AdamW · LR 1e-4 – 1e-3 · 30–50 epochs · best-checkpoint saving · early stopping (patience 5–10)

K Kushagra

Fatigue is a temporal condition — a single frame cannot separate a tired driver from one who is blinking or talking. The module therefore classifies 16-frame clips, not images.

CANDIDATES

CNN-LSTM · CNN-GRU · TCN · Lightweight 3D CNN

SELECTED

CNN-LSTM · input (16, 224, 224, 3)

WHY CNN-LSTM

- Best long-range temporal learning of the three sequence models tried
- Best overall performance among evaluated architectures

SPECIFICATION

Classes : Safe · Caution · High Risk

Loss : CrossEntropyLoss

Metrics : Acc · P · R · F1 · CM

Optim : Adam · 1e-4 · 4 · 5 ep

CNN Encoder → LSTM → Softmax



S Shiwani

Instead of raw pixels, this module reads five interpretable geometric signals per frame — the same evidence a human observer would use to say someone looks drowsy.

CANDIDATES

LSTM · GRU · TCN · MLP

SELECTED

LSTM · input (30, 5)

WHY LSTM

- EAR, MAR and head pose evolve over seconds, not frames
- Outperformed MLP, GRU and TCN on long-range dependency
- Tiny input tensor — cheapest module in the whole system

SPECIFICATION

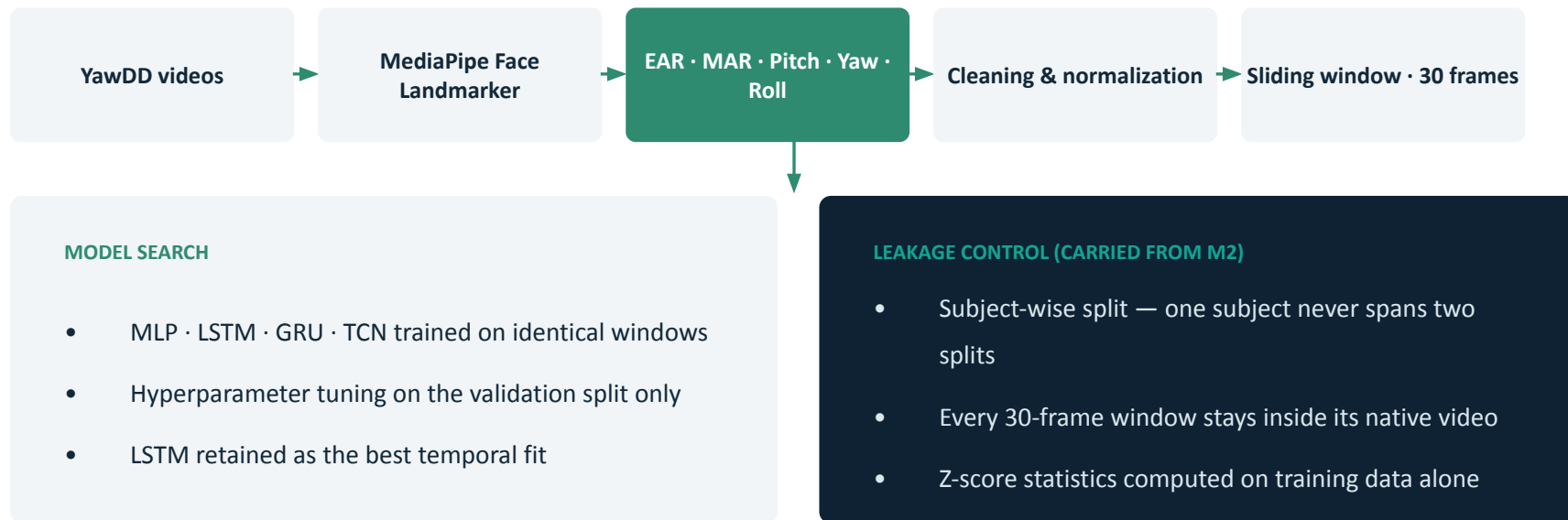
Features : EAR MAR Pitch Yaw Roll

Classes : Alert · Mild · Drowsy

Loss : CrossEntropyLoss

Optim : lr 0.001 · bs 32 · 30 ep

From Face Landmarks to Fatigue Class



S Shubham

Five-way image classification of what the driver's hands and gaze are doing: safe driving, texting, talking on the phone, turning, or other activity.

CANDIDATES

MobileNetV3 · ResNet50 · EfficientNet-B0

SELECTED

MobileNetV3 · input $224 \times 224 \times 3$

THE DELIBERATE TRADE-OFF

EfficientNet-B0 was the most accurate candidate at 93.98%. MobileNetV3 was still selected.

- Depthwise-separable blocks, SE modules, NAS-tuned — far fewer parameters
- Target hardware is Raspberry Pi / Jetson Nano, not a data-centre GPU
- ~12.5 ms per image keeps the whole five-model stack real-time
- A ~4-point accuracy gap costs less than missing the latency budget

The Only Module With Numbers Yet

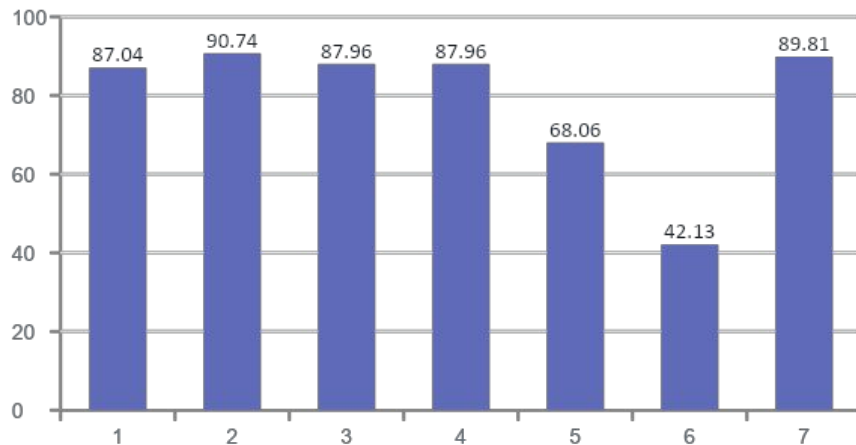
BASELINE · IMAGENET-PRETRAINED, CANDIDATE SUBSET

MODEL	ACCURACY	F1
MobileNetV3 (15 ep)	89.35%	89.39%
ResNet50 (10 ep)	92.13%	92.14%
EfficientNet-B0 (10 ep)	93.98%	93.98%

*Adam · lr 0.001 · batch 32 · identical stratified subset (4,200 images)
for every candidate.*

Large batches destabilised training; SGD converged far too slowly.

MobileNetV3 hyperparameter sweep (accuracy %)



Planned M4 configuration: Adam · lr 0.0005 · batch 32 → 90.74%



Compliance is a localisation problem, not a classification one — the system must say where the belt and the phone are, not merely whether the scene looks unsafe.

CANDIDATES

YOLOv8n · YOLO11n · YOLOv8s

SELECTED

YOLOv8n · **640 × 640** · **2 classes**

DETECTOR ANATOMY

CSP backbone → **PAN-FPN neck** → **anchor-free head**

Loss = Box loss + Classification loss + Distribution Focal Loss (DFL). Metrics: mAP@50, mAP@50-95, Precision, Recall.

WHY THE NANO VARIANT

- Fastest of the three on Jetson / Raspberry Pi class hardware
- ~12.5 ms per image, 4 GB GPU minimum
- Accuracy loss vs YOLOv8s is small on two large, well-separated classes

R Ravina

Localise and classify two unsafe behaviours live, so a downstream rule can raise a driver alert. Three Ultralytics detectors were fine-tuned under identical conditions to isolate two variables: generation and capacity.

CANDIDATES

YOLOv8n · YOLO11n · YOLOv8s

SELECTED

YOLOv8n · 640 × 640 · 2 classes

THREE DETECTORS, ONE TOOLCHAIN

YOLOv8n 3.01 M par · 8.2 GFLOPs

YOLO11n 2.59 M par · 6.4 GFLOPs

YOLOv8s 11.13 M par · 28.4 GFLOPs

Ultralytics ships v8 and v11 in one library — architecture effects are isolated from framework differences.

WHY THIS FAMILY

- Single forward pass — real-time on edge GPUs, unlike Faster R-CNN or DETR
- Anchor-free + DFL head copes with a thin cigarette and a tall bottle alike
- COCO-pretrained backbone, head re-initialised to nc = 2

A Fair Proxy, Then a Fair Fight

CANDIDATE SUBSET VS FULL DATASET

SET	IMAGES	BOXES / IMG	SMOKING
Full dataset	9,625	1.180	48.2%

Candidate subset 3,110 1.175 47.9%

- Stratified by dominant class; sampled inside existing train/val/test folders — no cross-split leakage
- 30% of train, 60% of val/test → 2,665 / 222 / 223 images
- Mean boxes per image differ by 0.005; class proportions by under half a point

Colab · Tesla T4 · Ultralytics 8.4.96 · PyTorch 2.11 + CUDA 12.8 · seed 42

BASELINE · DEFAULT CONFIG, 30 EPOCHS

MODEL	P	R	mAP@50	mAP@50-95
YOLOv8n	0.7678	0.7129	0.7476	0.3441
YOLO11n	0.7208	0.7143	0.7606	0.3340
YOLOv8s	0.7393	0.7118	0.7403	0.3346

Selected on the strict metric: YOLOv8n at 0.3441 mAP@50-95.

- AdamW auto-selected ($\text{lr} \approx 0.001667$); mosaic closed for the final 10 epochs
- COCO has no smoking or drinking class, so default-config fine-tuning — not the raw pretrained model — is the correct baseline

Capacity Did Not Buy Accuracy

3×

the parameters and compute in YOLOv8s bought no accuracy — it ties YOLO11n and trails YOLOv8n

0.76

mAP@50 for YOLO11n, the best on the loose metric, while YOLOv8n leads on Precision and mAP@50-95

~0.05

the gap by which Recall trails Precision across all three models — small cigarettes are occasionally missed

END-TO-END SETUP

Letterbox 640²

CSP backbone

PAN-FPN neck

Anchor-free DFL head

NMS → alert rule

Loss = CIoU (box) + BCE (class) + DFL (distribution). Val fold drives selection; the test fold stays held out.

Open for M4: retrain YOLOv8n on the full 9,625-image set with a longer schedule, target the Recall gap on small objects, and re-check the YOLOv8s accuracy-vs-latency trade-off against the real deployment hardware.

What Was Chosen, and Why

MODULE	CANDIDATES	SELECTED	INPUT	OUTPUT	PRIMARY REASON
Video Fatigue	CNN-LSTM, CNN-GRU, TCN, 3D CNN	CNN-LSTM	(16, 224, 224, 3)	3 classes	Temporal sequence modeling
Landmark Fatigue	LSTM, GRU, TCN, MLP	LSTM	(30, 5)	3 classes	Sequential facial feature analysis
Driver Activity	MobileNetV3, ResNet50, EfficientNet-B0	MobileNetV3	(224, 224, 3)	5 classes	Lightweight edge classification
Seat Belt & Phone	YOLOv8n, YOLO11n, YOLOv8s	YOLOv8n	640 × 640	2 classes	Fast, explainable localisation
Smoking / Drinking	YOLOv8n, YOLO11n, YOLOv8s	YOLOv8n	640 × 640	2 classes	Efficient real-time detection

Every module(Except Video Fatigue) runs on PyTorch or Ultralytics against an NVIDIA T4, in 4–16 GB of memory.

How Five Outputs Become One Score

● Driver Activity	Safe / Distracted	25%
● Video Fatigue	Safe / Caution / High Risk	25%
● Landmark Fatigue	Alert / Mild / Drowsy	25%
● Seat Belt	On / Off	15%
● Smoking / Drinking	Detected / Not detected	10%



RISK FUSION ENGINE

Wellness Score = $\Sigma (\text{Weight} \times \text{Module Risk})$

- Each module risk normalised to 0–1 before weighting
- Score reported on a 0–100 scale per trip
- Report carries per-module predictions, risk flags, trend history
- Delivered to the fleet dashboard (Uber / Ola / Rapido)

What Was Hard, and What Comes Next

CHALLENGES & LEARNINGS

- Accuracy alone is a misleading criterion once edge latency is a constraint
- Class imbalance and single-source classes limit what mAP can tell us
- Temporal modules need window-level, not frame-level, discipline to avoid leakage
- Five heterogeneous outputs must be normalised before they can be fused at all
- Keeping terminology and I/O contracts identical across five owners took real coordination

MILESTONE 4 PLAN

- Train all five modules on their prepared datasets
- Refine hyperparameters from the planned configurations
- Evaluate with per-module metrics and confusion matrices
- Implement and calibrate the Risk Fusion Engine weights
- Benchmark end-to-end latency on edge hardware
- Generate the first full Driver Wellness & Safety Report

The Architecture Is Settled.

Milestone 4 Is Implementation.

5

architectures selected
and justified

5

complete I/O, loss and
metric specifications

1

risk fusion design with
weights defined

0

modules left without a
compute budget

Questions

Kushagra · Shiwani · Shubham · Sohini · Ravina